Buffer Overflow y Explotación

© **Universidad de Palermo** | Prohibida la reproducción total o parcial de imágenes y textos.

UP
Universidad de Palermo

Vulnerabilidades

Las vulnerabilidades son la piedra angular de la seguridad, puesto que suponen el origen del que derivan numerosos fallos de seguridad.

Una vulnerabilidad en un programa informático o software es simplemente un error, un problema en su código o en su configuración.

Es muy probable –por no decir que se produce siempre– que los programas contengan errores, puesto que han sido creados por seres humanos.

Esto es especialmente frecuente en el caso de las aplicaciones muy complejas (como por ejemplo, un sistema operativo), que tienden a contener errores de manera exponencial.

La peculiaridad que convierte un simple fallo en una vulnerabilidad es la posibilidad de que el "abuso inteligente" de este defecto pudiera llevar a un riesgo de seguridad que compromete todo el sistema sobre el que se ejecuta esa aplicación.

UP Universidad de Palermo ©

¿Qué es una vulnerabilidad?

Una vulnerabilidad es un fallo en un programa o sistema informático. Pero no cualquiera, sino un fallo de seguridad.

Es necesaria esta distinción puesto que no todos los errores de programación derivan en fallos de seguridad.

Un error en un programa puede llevar a que no funcione correctamente o que su comportamiento no sea el esperado, pero no todos estos tipos de problemas pueden considerarse fallos de seguridad.

Según la capacidad de aprovecharse de este defecto, la vulnerabilidad será más o menos grave.

Una vulnerabilidad se define, básicamente, por **cinco factores** o parámetros que deben identificarla.

UP Universidad de Palermo ©

1. Producto

Para definir una vulnerabilidad, lo primero que es necesario conocer es a qué productos afecta.

Dentro de un mismo programa puede afectar a una sola versión, a toda una rama o incluso a programas totalmente diferentes que compartan un mismo fallo.

Este último supuesto ocurre cuando la aplicación afectada por la vulnerabilidad reside en sistemas operativos diferentes.

Por ejemplo,

- Todas las distribuciones de Linux comparten una buena cantidad de software, entre ellos, el propio núcleo del sistema.

Una sola vulnerabilidad en el kernel puede afectar a todas las distribuciones que lo utilicen, desde Debian a Mandriva pasando por OpenSuSE.

Para identificar correctamente una vulnerabilidad debe especificarse a qué productos o versiones de productos afecta concretamente.

UP Universidad de Palermo ©

2. Dónde

Dentro de un mismo programa, una vulnerabilidad se localiza habitualmente en un componente o módulo.

Los programas suelen componerse de varios módulos que interactúan entre sí.

Una vulnerabilidad puede encontrarse en un módulo concreto del programa o bien por utilizar una configuración concreta.

Por ejemplo:

- Puede existir una vulnerabilidad en el módulo de interpretación de ficheros en formato RTF en Microsoft Word sin afectar al módulo que procesa otro tipo de ficheros.
- O en el módulo de procesado de ficheros MP3 en el programa de reproducción Winamp.

Es posible que la vulnerabilidad no pueda ser aprovechada si este módulo no se encuentra activo.

Por ejemplo:

- El módulo de procesamiento de JavaScript en documentos PDF no se encuentra activo por defecto en Adobe Reader.

UP Universidad de Palermo ©

3. Causa y consecuencia

¿Cuál es el origen del problema?

¿En qué falló su programador?

Esto se refiere al fallo técnico concreto que cometió el programador a la hora de crear la aplicación que es el origen de la vulnerabilidad.

Por ejemplo:

- Puede que no comprobase bien qué valores alojaba una variable, los límites de la memoria o que olvidara establecer unos permisos adecuados a unos ficheros.
- Las consecuencias técnicas de estos fallos suelen ser diferentes.
- Desde el desbordamiento de memoria hasta el consumo excesivo de memoria.
- Estas consecuencias suelen ser siempre las mismas derivadas de los mismos descuidos, y es lo que buscarán los cazadores de vulnerabilidades.

3. Causa y consecuencia

Un ejemplo claro es que una falta de comprobación de caracteres en una aplicación web (causa), lleve a una posible inyección SQL (consecuencia).

Otro ejemplo:

- Una falta de comprobación de límites en una variable (causa), puede llevar a un desbordamiento de memoria intermedia (consecuencia).
- Un último ejemplo: un descuido a la hora de establecer los permisos de un servicio en Windows (causa) puede llevar a un salto de restricciones (consecuencia).

UP Universidad de Palermo ©

4. Impacto

El impacto es lo que puede conseguir un atacante que aprovechase la vulnerabilidad.

Por ejemplo:

- Si existe un desbordamiento de memoria intermedia, es posible que el atacante pueda conseguir ejecutar código.
- Si la consecuencia de la vulnerabilidad es que el programa comienza a consumir recursos, es posible que el atacante pueda llegar a conseguir una denegación de servicio (hacer que el programa deje de responder).
- Si no se han comprobado bien los permisos del programa (causa), puede que se produzca un salto de restricciones (consecuencia) y que el atacante consiga elevar privilegios en el sistema (impacto).

UP Universidad de Palermo ©

4. Impacto

El impacto define en gran medida la gravedad de la vulnerabilidad.

La ejecución de código arbitrario supone la mayor gravedad puesto que significa que el atacante podrá ejecutar cualquier programa en el sistema de su víctima.

Por tanto, podría realizar cualquier acción.

En estos casos, se dice que el sistema queda "comprometido" porque ha quedado en manos de la voluntad de un tercero.

UP Universidad de Palermo ©

5. Vector

A la forma que tiene el atacante de aprovechar la vulnerabilidad se le conoce como “vector de ataque”.

Un vector de ataque común es el envío de información especialmente manipulada a un puerto concreto del sistema.

Otra forma de conseguir aprovechar una vulnerabilidad es creando un fichero manipulado que será procesado por ese programa.

Por ejemplo:

- Si se encuentra una vulnerabilidad en Word, es muy probable que el vector de ataque sea un archivo en formato .doc que aproveche la vulnerabilidad.
- Si la víctima lo procesa con un Word vulnerable, el atacante conseguirá el impacto deseado.

UP Universidad de Palermo ©

5. Vector

Otros vectores de ataque pueden ser muy sencillos de llevar a cabo, como hacer que la víctima visite un enlace.

Por ejemplo:

- Muchas de las vulnerabilidades encontradas en los navegadores son aprovechadas por atacante creando una página web adulterada que, al ser visitada con el navegador vulnerable, aprovecha la vulnerabilidad.
- Por tanto, enviar un enlace a la potencial víctima sería el vector de ataque en este caso y el impacto podría ser la ejecución de código.

UP Universidad de Palermo ©

La aplicación de diseño gráfico MSPaint necesita conocer el tamaño de la imagen que va a abrir antes de procesarla.

Para conseguir esto, mspaint.exe se ayuda de fichero llamado image_size.dll que está especializado en esta función.

El programador de la aplicación MSPaint 7 omitió la comprobación correcta de un parámetro que define la longitud que debe tener una imagen, en la función SetImageSize del fichero image_size.dll.

Así pues, como no se comprueba que el valor del parámetro coincida con el tamaño real de la imagen, se produce un desbordamiento de memoria.

Explicado de forma sencilla:

- La memoria del ordenador llega a sitios donde no debería.
- Por tanto, un atacante puede retocar la cabecera de una imagen para modificar su atributo de tamaño, añadir a esa imagen el código que se quiere ejecutar y aprovecharse de la vulnerabilidad.

Buffer Overflow

Desbordamiento de buffer

Un desbordamiento de buffer (del inglés *buffer overflow* o *buffer overrun*) es un error de software que se produce cuando un programa no controla adecuadamente la cantidad de datos que se copian sobre un área de memoria reservada a tal efecto (buffer)

De forma que si dicha cantidad es superior a la capacidad preasignada los bytes sobrantes se almacenan en zonas de memoria adyacentes, sobrescribiendo su contenido original.

Esto constituye un fallo de programación.

En las arquitecturas comunes de computadoras no existe separación entre las zonas de memoria dedicadas a datos y las dedicadas a programa, por lo que los bytes que desbordan el buffer podrían grabarse donde antes había instrucciones, lo que implicaría la posibilidad de alterar el flujo del programa, llevándole a realizar operaciones imprevistas por el programador original.

Esto es lo que se conoce como una vulnerabilidad.

UP Universidad de Palermo ©

Una vulnerabilidad puede ser aprovechada por un usuario malintencionado para influir en el funcionamiento del sistema.

En algunos casos, el resultado es la capacidad de conseguir cierto nivel de control saltándose las limitaciones de seguridad habituales.

Si el programa con el error en cuestión tiene privilegios especiales, constituye en un fallo grave de seguridad.

Se denomina shellcode al código ejecutable especialmente preparado que se copia al host objeto del ataque para obtener los privilegios del programa vulnerable.

La capacidad de los procesadores modernos para marcar zonas de memoria como protegidas puede usarse para aminorar el problema.

Si se produce un intento de escritura en una zona de memoria protegida, se genera una excepción del sistema de acceso a memoria, seguido de la terminación del programa.

Por desgracia, para que esta técnica sea efectiva los programadores han de indicar al sistema operativo las zonas que se necesita.

UP Universidad de Palermo ©

Descripción técnica

- Un desbordamiento de buffer ocurre cuando los datos que se escriben en un buffer corrompen aquellos datos en direcciones de memoria adyacentes a los destinados para el buffer, debido a una falta de validación de los datos de entrada.

- Esto se da comúnmente al copiar cadenas de caracteres de un buffer a otro.

UP Universidad de Palermo ©

Ejemplo básico

En este ejemplo, un programa tiene definidos dos elementos de datos continuos en memoria:

- Un buffer de 8 bytes tipo string, A.
- Otro de dos bytes tipo entero, B.

Al comienzo, A contiene bytes nulos y B contiene el número 3 (cada carácter se representa mediante un byte).

0	0	0	0	0	0	0	0	3
BuA								BuB

Ejemplo básico

A continuación, el programa intenta almacenar la cadena de caracteres "demasiado" en el buffer A, seguido de bytes nulos para marcar el fin de string.

Al no validarse la longitud de la cadena, se sobrescribe el valor de B:

'd'	'e'	'm'	'a'	's'	'i'	'a'	'd'	'o'
BuA								BuB

A pesar de que el programador no quería cambiar el contenido del buffer B, el valor de este ha sido reemplazado por un número equivalente a parte de la cadena de caracteres.

Si B fuese la única variable aparte de A definida en el programa, la escritura de datos que sobrepasen los límites de B generarían un error como segmentation fault, concluyendo así el programa.

UP Universidad de Palermo ©

Código fuente de ejemplo

En el siguiente ejemplo se presenta un código fuente en C con un error de programación.

Una vez compilado, el programa generará un desbordamiento de buffer si se lo invoca desde la línea de comandos con un argumento lo suficientemente grande, pues este argumento se usa para llenar un buffer, sin validar previamente su longitud.

UP Universidad de Palermo ©

```
/* overflow.c  demuestra un desbordamiento de buffer */
#include <stdio.h>
#include <string.h>
int main(int argc, char *argv[])
{
char buffer[10];
if (argc < 2)
{
fprintf(stderr, "MODO DE USO: %s string\n", argv[0]);
return 1;
}
strcpy(buffer, argv[1]);
return 0;
}
```

UP Universidad de Palermo ©

Este programa puede reescribirse en forma más segura usando la función **strncpy** de la siguiente manera:

```
#include <string.h>
#include <stdio.h>
int main(int argc, char *argv[])
{
char buffer[10];
if (argc < 2)
{
fprintf(stderr, "MODO DE USO: %s string\n", argv[0]);
return 1;
}
strncpy(buffer, argv[1], sizeof(buffer));
buffer[sizeof(buffer)  1] = '\0';
return 0;
}
```

Explotación

Explotación

Partiendo de las vulnerabilidades descubiertas en la fase anterior, tendremos que dedicarle un tiempo a la investigación de las herramientas y técnicas para explotar las mismas.

Una vez encontrado estos exploits, procederemos a explotar las vulnerabilidades.

UP Universidad de Palermo ©

Explotación

¿Es necesario explotar todas las vulnerabilidades que se encuentren en la fase de enumeración?

No, debemos explotar aquellas vulnerabilidades que nos permitan llegar a cumplir con nuestro objetivo.

Al ser esta una tarea intrusiva, por más que se haga de manera controlada, debe quedarle claro al cliente de las acciones que llevaremos en esta fase y en nuestro NDA firmado es donde se especifica dicha cuestión.

UP Universidad de Palermo ©

Exploit Database

Exploit Database es mantenida por Offensive Security y es un proyecto sin fines de lucro que se ofrece como un servicio público de seguridad ofensiva.

UP Universidad de Palermo ©

Exploit DB por consola

```
root@kali:~# searchsploit eternalblue
------------------------------------------------------------------------------------------------- ---------------------------------
 Exploit Title                                                                                   |  Path
------------------------------------------------------------------------------------------------- ---------------------------------
Microsoft Windows 7/2008 R2 - 'EternalBlue' SMB Remote Code Execution (MS17-010)                 | windows/remote/42031.py
Microsoft Windows 7/8.1/2008 R2/2012 R2/2016 R2 - 'EternalBlue' SMB Remote Code Execution (MS17-010) | windows/remote/42315.py
Microsoft Windows 8/8.1/2012 R2 (x64) - 'EternalBlue' SMB Remote Code Execution (MS17-010)       | windows_x86-64/remote/42030.py
------------------------------------------------------------------------------------------------- ---------------------------------
Shellcodes: No Results
```

Búsqueda de Exploits

Existen varios sitios web en los que podemos buscar vulnerabilidades y exploits para el software que hayamos identificado a través de cualquiera de las técnicas anteriormente mencionadas.

Nota: debemos tener mucho cuidado al descargar y ejecutar código tanto en nuestros equipos como en los equipos de la organización a analizar, porque los mismos podrían contener código malicioso.

A continuación podemos encontrar una lista de sitios web en los cuales encontraremos exploits y otros recursos de interés utilizados para la explotación de vulnerabilidades:

UP Universidad de Palermo ©

http://www.securityfocus.com



UP Universidad de Palermo ©

http://packetstormsecurity.com



http://www.cvedetails.com

CVE Details

The ultimate security vulnerability datasource

(e.g.: CVE-2009-12

g In Register

Switch to https://
Home

Browse :
Vendors
Products
Vulnerabilities By Date
Vulnerabilities By Type

Reports :
CVSS Score Report
CVSS Score Distribution

Search :
Vendor Search
Product Search
Version Search
Vulnerability Search
By Microsoft References

Top 50 :
Vendors
Vendor Cvss Scores
Products
Product Cvss Scores
Versions

Other :
Microsoft Bulletins
Bugtraq Entries
CWE Definitions

Enter a CVE id, product, vendor, vulnerability type...

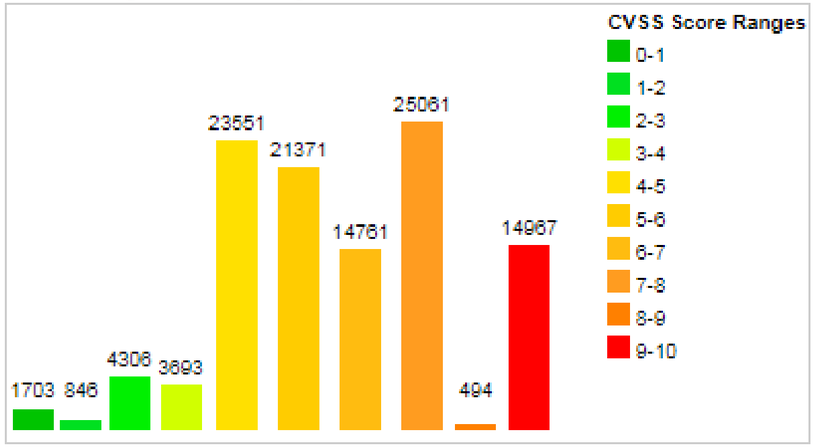
Current CVSS Score Distribution For All Vulnerabilities

Distribution of all vulnerabilities by CVSS Scores

CVSS Score	Number Of Vulnerabilities	Percentage
0-1	1703	1.50
1-2	846	0.80
2-3	4306	3.90
3-4	3693	3.30
4-5	23551	21.30
5-6	21371	19.30
6-7	14761	13.30
7-8	25061	22.60
8-9	494	0.40
9-10	14967	13.50
Total	110753	

Weighted Average CVSS Score: **6.6**

Vulnerability Distribution By CVSS Scores



http://cxsecurity.com



http://shell-storm.org/shellcode/

Blog Repository Triton, our dynamic binary analysis framework Diary of a reverse-engineer



- Twitter Profile
- GitHub Profile
- Google+ Profile
- Linkedin Profile
- RSS feeds
- Email

Shellcodes database for study cases

Description

Although these kinds of shellcode presented on this page are rarely used for real exploitations, this page lists some of them for study cases and proposes an API to search specific ones. Thanks all for your contributions of this database but we stopped to accept shellcodes. To learn modern exploitation, checkout how to the Return Oriented Programming works.

API

This is very straightforward to communicate with this API. Just send a simple **GET** method. The "**s**" argument contains your keyword.

```
http://shell-storm.org/api/?s=<keyword>
```

Use "*" for multiple keywords search.

```
/?s=<keyword1>*<keyword2>*<keyword3>
```

The output should be like this:


EXPLOIT DATABASE
blog
exploit
Currently Archiving 32438 Exploits
Updated (CVE And Archive): Thu Mar 26 2015
HOME
GHDB
ABOUT
REMOTE
LOCAL
WEB
DOS
SHELLCODE
PAPERS
SEARCH
SUBMIT
The Exploit Database
The Exploit Database (EDB) – an ultimate archive of exploits and vulnerable software. A great resource for penetration testers, vulnerability researchers, and security addicts alike. Our aim is to collect exploits from submittals and mailing lists and concentrate them in one, easy to navigate database.
Remote Exploits
Date D A V Description Plat. Author
2015-03-26 - QNAP admin shell via Bash Environment Variable Code Injection hardware Patrick Pellegrin
2015-03-26 - QNAP Web Server Remote Code Execution via Bash Environment Variable Code Injection hardware Patrick Pellegrin
2015-03-26 WebGate eDVR Manager Stack Buffer Overflow windows Praveen Darshanam
2015-03-25 - Adobe Flash Player Arbitrary Code Execution windows SecurityObscurity



© Universidad de Palermo
Prohibida la reproducción total o parcial de imágenes y textos.